

Boulder Dash 1984 Remake

Lukáš Bien

ČVUT-FIT

bienluka@fit.cvut.cz

4. ledna 2026

1 Úvod

1.1 Cíle

Cílem semestrální práce bylo vytvořit hratelný remake hry *Boulder Dash* (1984) v jazyce Python s využitím knihovny `pyglet` [1]. Projekt je implementován jako dlaždicová hra s tick systémem — fyzika a chování objektů se vyhodnocuje periodicky.

Hlavní cíle projektu:

- implementovat mechaniky hry — pohyb hráče, posouvání kamenů, sbírání drahokamů, pohyb nepřátel. . .
- implementovat gravitaci pro kameny a drahokamy
- uživatelské rozhraní — zobrazení skóre, počet nasbíraných drahokamů, zbývající čas. . .
- načítání úrovní (levelů) z textových souborů

Projekt je určen pro vzdělávací, nekomerční použití. Použité zdroje grafiky, fontu a dat úrovní jsou uvedeny v souboru `assets/CREDITS.txt`.

2 Vstupní data

2.1 Formát úrovní

Úrovně jsou uloženy ve formátu TOML v adresáři `assets/levels/`. Každý soubor obsahuje meta informace (např. název, časový limit, počet požadovaných drahokamů) a ASCII mapu dlaždic. Při načtení se jednotlivé znaky mapují na interně definované typy dlaždic.

Konfigurace hry je uložena v YAML souboru `game/cfg/config.yaml` a načítá se pomocí knihovny PyYAML [3].

2.2 Původ dat

Základní rozložení urovní vychází z dekodovaných dat původní verze (C64) publikovaných Jake Gordonem [4]. Grafický spritesheet je převzat ze stejného zdroje a byl barevně upraven [5]. Použitý font „Boulder Dash 6128“ pochází z portálu dafont [6].

```
assets > levels > cave4.toml
1  [meta]
2  name = "Cave D: Butterflies"
3  time_limit = 120
4  diamonds_required = 36
5  diamond_value = 5
6  bonus_diamond_value = 20
7
8  [colors]
9  background1 = "orange"
10 background2 = "purple"
11 foreground = "white"
12
13 [level]
14 data = ""
15
16 WXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
17 W.....P.....P.....P.....P.....P.....W
18 W.....P.....P.....P.....P.....P.....W
19 W.....P.....P.....P.....P.....P.....W
20 W.....P.....P.....P.....P.....P.....W
21 W.....P.....P.....P.....P.....P.....W
22 W.....P.....P.....P.....P.....P.....W
23 W.....P.....P.....P.....P.....P.....W
24 W.....P.....P.....P.....P.....P.....W
25 W.....P.....P.....P.....P.....P.....W
26 W.....P.....P.....P.....P.....P.....W
27 W.....P.....P.....P.....P.....P.....W
28 W.....P.....P.....P.....P.....P.....W
29 W.....P.....P.....P.....P.....P.....W
30 W.....P.....P.....P.....P.....P.....W
31 W.....P.....P.....P.....P.....P.....W
32 W.....P.....P.....P.....P.....P.....W
33 W.....P.....P.....P.....P.....P.....W
34 W.....P.....P.....P.....P.....P.....W
35 W.....P.....P.....P.....P.....P.....PW
36 WXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
37 ""
```

Obrázek 1: Příklad TOML souboru popisující jednu úroveň.

3 Metody/postupy/algoritmy

3.1 Architektura

Aplikace je rozdělena na několik hlavních částí:

- `game/game.py`: řízení stavu hry (skóre, životy, čas, přechod úrovní), zpracování vstupu.
- `game/lvl/level.py`: načítání úrovní a herní simulace (fyzika, nepřátele...).
- `game/gfx/renderer.py`: vykreslování dlaždic a uživatelského rozhraní.
- `game/cfg/config.py`: načítání konfigurace.

3.2 Simulace herní fyziky

Fyzika kamenů a drahokamů je vyhodnocována po částech. Pro každou aktualizaci se nejprve vytvoří kopie typů dlaždic a stavů „falling“ a poté se aplikuje sada pravidel:

- volný pád do prázdné buňky,
- rozdělení „falling“ vs. „stojící“ objekt (např. zabití hráče jen při dopadu padajícího objektu),
- sklouzávání z kulatých povrchů (pravidla pro „rounded“ dlaždice),
- zabití nepřátel padajícím objektem (butterfly vytvoří shluk drahokamů).

3.3 Nepřátele

V projektu jsou implementovány dva typy nepřátel: *firefly* a *butterfly*. Oba se pohybují po mřížce podle pravidla „následování stěny“ (wall following). Kontakt s hráčem vede k jeho smrti.

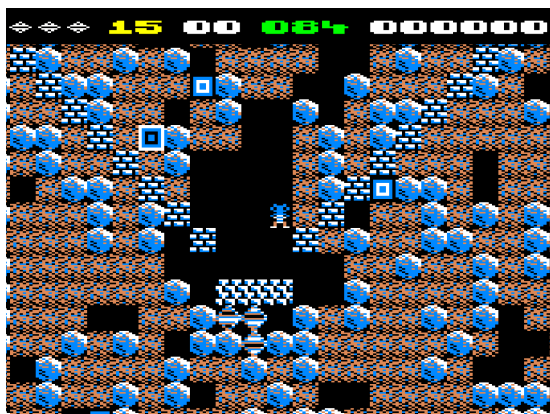
Butterfly má specifické chování při smrti: vytvoří ve svém okolí 3×3 drahokamů.

3.4 Amoeba

Amoeba se náhodně rozrůstá do sousedních prázdných nebo „dirt“ buněk s určitou pravděpodobností. Pokud se amoeba „uzavře“ (nemá kam růst), transformuje se na drahokamy. Pokud překoná maximální velikost, transformuje se na kameny.

3.5 Magic wall

Magic wall (magická stěna) transformuje padající objekty: kámen $\rightarrow$ diamant a diamant $\rightarrow$ kámen. Transformace probíhá jen v případě, že je výstupní buňka pod stěnou volná.



Obrázek 2: Magic wall (třetí buňka pod hráčem).

3.6 Testování

Projekt obsahuje jednotkové testy spustitelné příkazem `pytest` [2]. Pro testy je hra schopna běžet v „headless“ režimu bez vytvoření okna, aby testy běžely rychle.

4 Výsledky

Výsledkem je funkční hra s 20 úrovněmi, základním uživatelským rozhraním a implementovanými mechanikami popsány v předchozích sekcích. Při prohře se zobrazí „game over“ s finálním získaným počtem skóre a hru lze restartovat podržením klávesy „R“.



Obrázek 3: Ukázka hry.



Obrázek 4: Konec hry s finálním skóre.

5 Závěr

Projekt naplnil cíl vytvořit remake hry Boulder Dash s klíčovými herními mechanikami. Mezi možná budoucí vylepšení patří zlepšení pravidel tak, aby více odpovídala originálu, vylepšení uživatelského rozhraní, či opravení různých vizuálních chyb. Projekt také může sloužit jako jednoduchá ukázka jak použít knihovnu `pyglet` [1].

Reference

- [1] `pyglet`: A cross-platform windowing and multimedia library for python. Online. [cit. 2026-01-04] <https://pyglet.readthedocs.io/>.
- [2] `pytest`: helps you write better programs. Online. [cit. 2026-01-04] <https://docs.pytest.org/>.

- [3] pyyaml: Yaml parser and emitter for python. Online. [cit. 2026-01-04] <https://pyyaml.org/>.
- [4] Jake Gordon. Boulder dash decoded cave data (pdf). Online, 2011. [cit. 2026-01-03] https://jakesgordon.com/writing/javascript-boulderdash/decoded_cave_data.pdf.
- [5] Jake Gordon. Boulderdash rendering (spritesheet). Online, 2011. [cit. 2026-01-03] <https://jakesgordon.com/writing/boulderdash-rendering/sprites.png>.
- [6] Damien Gosset. Boulder dash 6128 font. Online (dafont.com), 2020. [cit. 2025-12-10] <https://www.dafont.com/boulderdash6128.font>.